

June 18th, 2024

Timer Output Assignment Report

Embedded System 3, LAB 04

Document Version 1.0

Edited by:

Farros Ramzy (3767353)

Abstract

This technical report describes about the use of timer bit registers in the STM32 board to write an output from the STM32 Nucleo board. This report focused on the procedure, context, and knowledge about the timers output registers and PWM pins of the STM32-F303RE board microcontroller.

List of Figures

Figure 2. 1. 1 STM32-F303RE Board pinout.	2
Figure 2. 1. 2 System Clock Configuration.	3
Figure 2. 2. 1 LED PWM output wiring diagram.	3
Figure 2. 2. 2 MODER setup guide table.	4
Figure 2. 2. 3 GPIOB pin 3 setup.	4
Figure 2. 2. 4 Alternate Function Register Low guide table.	5
Figure 2. 2. 5 AFR GPIOB setup.	5
Figure 2. 2. 6 APB1 RCC register guide table.	6
Figure 2. 2. 7 Timer 2 Pre-scaler and Auto Reload setup.	6
Figure 2. 2. 8 Channel setup.	7
Figure 2. 2. 9 Brightness value as private define.	7
Figure 2. 3. 1 Timer config with interrupts.	8
Figure 2. 3. 2 Interrupts enabler.	8
Figure 2. 4. 1 PWM Servo wiring diagram.	9
Figure 2. 4. 2 Alternate Function Register High guide table.	10
Figure 2. 4. 3 GPIO setup on PA9 with AFRH register.	10
Figure 2. 4. 4 Setting up ARR to 20 ms, representing 50 Hz timer tick.	11
Figure 2. 4. 5 Solution to set the PWM output of the servo due to its angular scale.	11
Figure 3. 1 First PWM result on Servo Pulse.	12
Figure 3. 2 Second PWM result on Servo Pulse.	12
Figure 3. 3 Third PWM result on Servo Pulse.	12

Table of Contents

1.	Introduction	1
1.1.	Equipment.....	1
2.	Procedure.....	2
2.1.	Preparation	2
2.2.	Part A, Controlling LED PWM Output	3
2.3.	Part B, Pulsing LED Output with PWM Setting	8
2.4.	Part C, Controlling PWM Servo Motor.....	9
3.	Experiment Result.....	12
4.	Conclusion.....	13
5.	Recommendation.....	13
6.	References	14

1. Introduction

This assignment is meant to understand the concept of the timer output and how the PWM mode of the STM32 Nucleo works. The assignment is starting from the way of configuring the timer system, the alternate functions of the GPIO pin registers, and also the PWM output.

The document will explain each part in the Procedure section, the result will be placed in the Conclusion section, and at least, a recommendation will be made in the Recommendation section. Some references that have been used for the practicum and included in this document are listed in the References part by the end of this report.

1.1. Equipment

The equipment needed to do this research practicum are:

1. A programming platform for the STM32 board (STM32CubeIDE)
2. An STM32-F303RE board
3. One LED
4. One PWM 90° Servo
5. A breadboard
6. A bunch of wires
7. One 470 Ω , $\pm 5\%$ resistor for one of the LED

2. Procedure

2.1. Preparation

To start the assignment, it is required to read the GPIO registers in the reference manual of the STM32-F303RE board which is listed in this References section.

After that, to understand the pin position that will be used from this STM32 board, the Nucleo User Manual document must be read.

From that manual, it can be known that the STM32-F303RE pins that can be used are presented in this Figure 2. 1. 1 below.

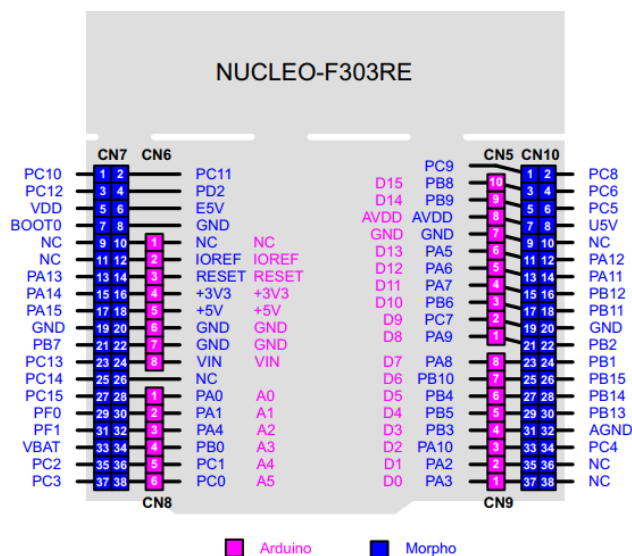


Figure 2. 1. 1 STM32-F303RE Board pinout.

From this Figure 2. 1. 1, it is visible that the pink or purple pins from the board is usable for the Arduino programming.

However, in this practicum, the pins that can be used for the STM GPIO register manipulation are the Morpho pins, reserved for the Nucleo STM32 board.

Nevertheless, there are some pins that cannot be used for external interference such as the internal LED pin, PA5 or PB13, and the internal button, PC13 because these pins are internally built in for the board.

After decided the output pin, the tools such as an LED, a resistor, and some jumper wires must be prepared.

After that, it is recommended that to read the Nucleo F303RE reference manual and the datasheet to locate the PWM pin registers on the microcontroller, then figuring out on how to use them.

If everything is prepared, then it is time to prepare the clock configuration of the system.

This clock configuration can be done in the (ioc) file of the STM project IDE, shown in this Figure 2. 1. 2 below.

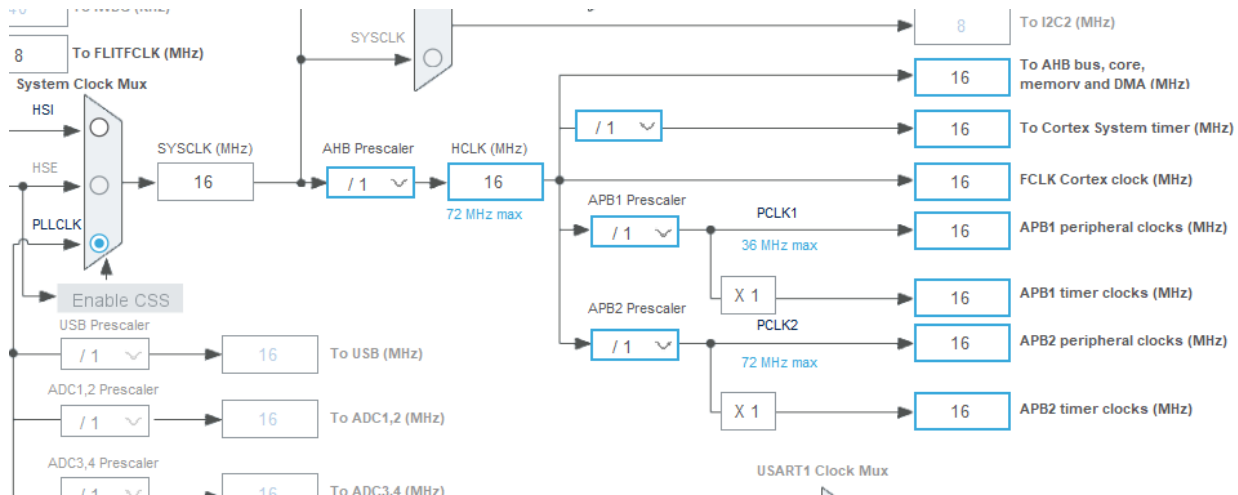


Figure 2. 1. 2 System Clock Configuration.

In this Figure 2. 1. 2 above the system clock is setup to 16 MHz, following to the assignment requirement, and the AHB clock set with no division, since the system will only use the peripheral bus with the timer interrupts to controls the output pins of the system.

2.2. Part A, Controlling LED PWM Output

There are some important steps to do the PWM setup for an output pin register. And in this case, an LED is chosen to be the output tool for the very basic PWM experiment.

To start with the assignment, it is better to determine the pin that will be used for the LED. And in this case, as what can be seen in this Figure 2. 2. 1 below, PB3 or pin 3 of GPIO port B is chosen for it.

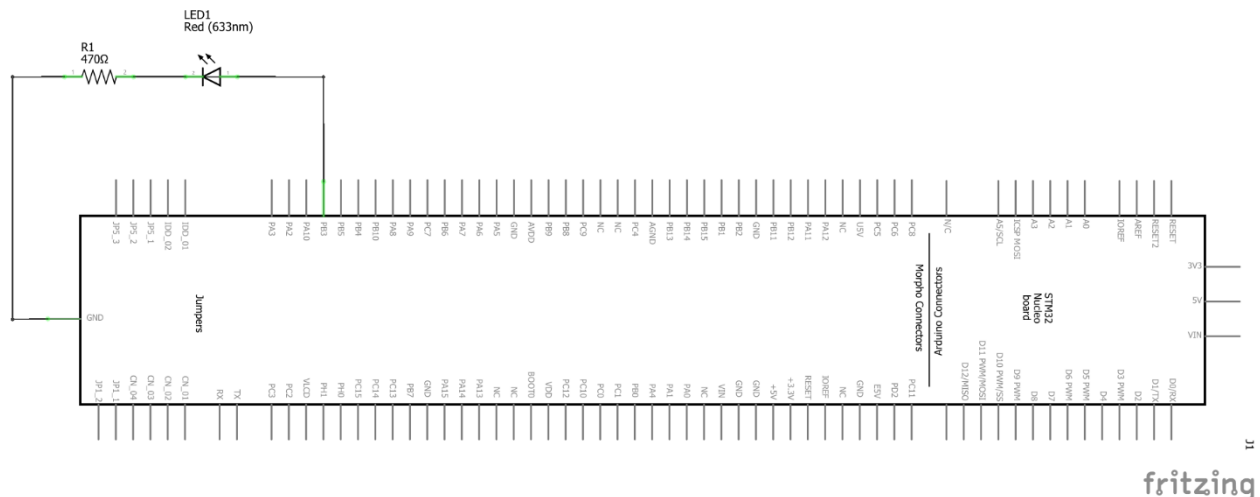


Figure 2. 2. 1 LED PWM output wiring diagram.

This wiring method on Figure 2. 2. 1 above will also be applied for part B of the assignment later.

After decides the wiring method, now is the time to setup the registers. The initialization of the GPIO pin is almost the same as the normal GPIO input and output way. However, the PWM output requires the connected pin tool to be setup as an alternate function.

The alternate function register of the pinout can be setup in the MODER register as 0b10, shown in this Figure 2. 2. 2 below.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits $2y+1:2y$ **MODERy[1:0]**: Port x configuration bits ($y = 0..15$)

These bits are written by software to configure the I/O mode.

00: Input mode (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Figure 2. 2. 2 MODER setup guide table.

And in the code, it will be applied like this Figure 2. 2. 3 below.

```

232 //sets the LED's pin MODE to 0b10 as an alternated output.
233 GPIOB->MODER |= GPIO_MODER_MODER3_1;|
234 //sets the LED's pin SPEED to medium.
235 GPIOB->OSPEEDR |= GPIO_OSPEEDER_OSPEEDR3_0;
236 //resets the LED's PUPD register.
237 GPIOB->PUPDR &= ~GPIO_PUPDR_PUPDR3;
238

```

Figure 2. 2. 3 GPIOB pin 3 setup.

After the MODER, SPEED, and Pull-up/Pull-down resistor are correctly setup, it is time to look at the register map in the STM32 Nucleo F303RE Datasheet to find the correct Alternate Function register that must be fit with pinout 3 of Port B GPIO. This Alternate Function also must connect to one of the timer registers of the Nucleo, including its port Channel.

And in this case, **AF1** (Alternate Function 1) is happened to be the pin map along these registers, which connect **PB3** to **Timer 2** in **Channel 2** of the Nucleo board.

Now, looking back at the reference manual, the AF1 is included in the low register of the Alternate Function (AFRL) like what is shown in this Figure 2. 2. 4 below.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR7[3:0]				AFR6[3:0]				AFR5[3:0]				AFR4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR3[3:0]				AFR2[3:0]				AFR1[3:0]				AFR0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRy[3:0]**: Alternate function selection for port x pin y (y = 0..7)
 These bits are written by software to configure alternate function I/Os

AFRy selection:

0000: AF0	1000: AF8
0001: AF1	1001: AF9
0010: AF2	1010: AF10
0011: AF3	1011: AF11
0100: AF4	1100: AF12
0101: AF5	1101: AF13
0110: AF6	1110: AF14
0111: AF7	1111: AF15

Figure 2. 2. 4 Alternate Function Register Low guide table.

Therefore, the setup of the pinout should use the AFRL (AFR[0] in code mapping) with the 0b0001 as its value like in this Figure 2. 2. 5 below.

```

239 //resets the low alternate function for pin PB3.
240 GPIOB->AFR[0] &= ~GPIO_AFRL_AFRL3;
241 //activate alternate function 1 to use timer 2 and channel 2 for PB3.
242 GPIOB->AFR[0] |= (0b0001 << GPIO_AFRL_AFRL3_Pos);
243 }
```

Figure 2. 2. 5 AFR GPIOB setup.

After the pin is correctly configured, now is the time to setup the related timer and the channel of the GPIO, to control the PWM value.

To do this, the first thing needed is to enable the Timer 2 in APB Register. And as can be seen in this Figure 2. 2. 6 below, Timer 2 is placed on pin 0 of the APB Register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	I2C3 RST ⁽¹⁾	DAC1 RST	PWR RST	Res	DAC2R ST	CAN RST	Res	USB RST	I2C2 RST	I2C1 RST	UART5 RST	UART4 RST	USART3 RST	USART2 RST	Res
		rw	rw		rw	rw		rw	rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 RST	SPI2 RST	Res	Res	WWDG RST	Res	Res	Res	Res	Res	TIM7 RST	TIM6 RST	Res	TIM4 RST	TIM3 RST	TIM2 RST
rw	rw			rw						rw	rw		rw	rw	rw

Figure 2. 2. 6 APB1 RCC register guide table.

This makes the setup of the register clock control easier since the number to set in this register is only 1.

After that, the timer must be put on reset first by using the clear bit on the timer enabler (pin CR1) to make sure that the system will not use the other setup that might be placed on the memory before this practicum.

And then, since the system uses 16 MHz, shown in this Figure 2. 1. 2 above, the pre-scaler and the auto reload register must be calculated to use the 16 MHz per reload period.

```
//prescale the system using 16MHz frequency.
TIM2->PSC = 16 - 1;
//set the PWM period in 1 ms.
TIM2->ARR = 1000 - 1;
```

Figure 2. 2. 7 Timer 2 Pre-scaler and Auto Reload setup.

In this Figure 2. 2. 7 above, the auto reload register is being setup to reload per 1 millisecond since the system will calculate the clock in millisecond range.

After that, the counter of the Timer 2 can be reset to 0 using the CNT register, and the Timer 2 is safe to be enabled.

Once the Timer 2 has been enabled, the channel of the Timer register can be setup properly.

There are three things that must be done on the channel setup for the output PWM pin register of the STM32 Nucleo board.

The first one is to enable the capture compare register pin related to the channel, clean bits the channel mode of the pins, reset the interrupt since it is not being used in this first part of the assignment, enabling the preload, and put the PWM value on the **CCR channel 2** register to emit the LED to the desired brightness level.

This method can be seen in this Figure 2. 2. 8 below.

```

195 void Channel2_Init(void) {
196     //set enable the compare register of channel 2 in timer 2.
197     TIM2->CCER |= TIM_CCER_CC2E;
198
199     //clean bits the channel 2 in timer 2 register.
200     TIM2->CCMR1 &= ~TIM_CCMR1_CC2S;
201     TIM2->CCMR1 &= ~TIM_CCMR1_OC2M;
202     TIM2->CCMR1 &= ~TIM_CCMR1_OC2PE;
203
204     //reset interrupt in channel 2 (interrupt is not used in this project).
205     TIM2->DIER &= ~TIM_DIER_CC2IE;
206
207     //set the PWM in channel 2 of timer 2 as the output compare mode.
208     TIM2->CCMR1 |= (TIM_CCMR1_OC2M_1 | TIM_CCMR1_OC2M_2);
209     //enable the preload.
210     TIM2->CCMR1 |= TIM_CCMR1_OC2PE;
211
212     //set the output range to 75% (750 from 1000 (the PWM period)).
213     TIM2->CCR2 = BRIGHTNESS;
214 }

```

Figure 2. 2. 8 Channel setup.

And since the system use 0 to 1000 range for the PWM output period, and the assignment asked for the 75% output of brightness, 750 value can be written on the **TIM2->CCR2** equation, or to make it more maintainable, using a private define value such as shown in this Figure 2. 2. 9 below can be helpful.

```

34 /* Private define -----
35 /* USER CODE BEGIN PD */
36 #define BRIGHTNESS 750
37 /* USER CODE END PD */
38

```

Figure 2. 2. 9 Brightness value as private define.

2.3. Part B, Pulsing LED Output with PWM Setting

In this part, almost all the setup procedure the same with the previous part of the assignment since the pinout GPIO is also PB3. However, there is a small little detail of change happen on the timer output settings.

Because the requirement of this part of the assignment is to make the LED output to pulse, **interrupt counter** must be added on the **Timer 2** and **Channel 2** setup. In this Figure 2. 3. 1, the interrupt calls on Timer 2 is being enabled by using the **TIM2_IRQn** pin on **NVIC** register.

```
172 /* USER CODE BEGIN 4 */
173 void Timer2_Init(void) {
174     //Activate timer 2 in APB register.
175     RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
176
177     //disable the timer.
178     TIM2->CR1 &= ~TIM_CR1_CEN;
179
180     //prescale the system using 16MHz frequency.
181     TIM2->PSC = 16 - 1;
182     //set the PWM period in 1 ms.
183     TIM2->ARR = 1000 - 1;
184
185     //configure NVIC for TIM2 interrupt.
186     NVIC_EnableIRQ(TIM2_IRQn);
187     NVIC_SetPriority(TIM2_IRQn, 0);
188
189     //reset counter of Timer 2 to 0.
190     TIM2->CNT &= ~TIM_CNT_CNT;
191
192     //enable the timer.
193     TIM2->CR1 |= TIM_CR1_CEN;
194 }
```

Figure 2. 3. 1 Timer config with interrupts.

And on the channel setup, the DMA interrupt register on the desired channel pin is being enabled, as what is shown in this Figure 2. 3. 2 below.

```
205 //set interrupt enable register in channel 2.
206 TIM2->DIER |= TIM_DIER_CC2IE;
```

Figure 2. 3. 2 Interrupts enabler.

Now using the interrupt handler of the Timer 2, the system can pulse the PWM output to get brighter and dimmer in each interrupt flag read.

2.4. Part C, Controlling PWM Servo Motor

This part of the assignment is not that different from the first two part. The differences are the value of the reload counter setup and the GPIO pin configuration.

To start on the assignment, it is necessary to attach the PWM servo to the correct pin of the STM32 Nucleo F303RE such as shown in this Figure 2. 4. 1 below.

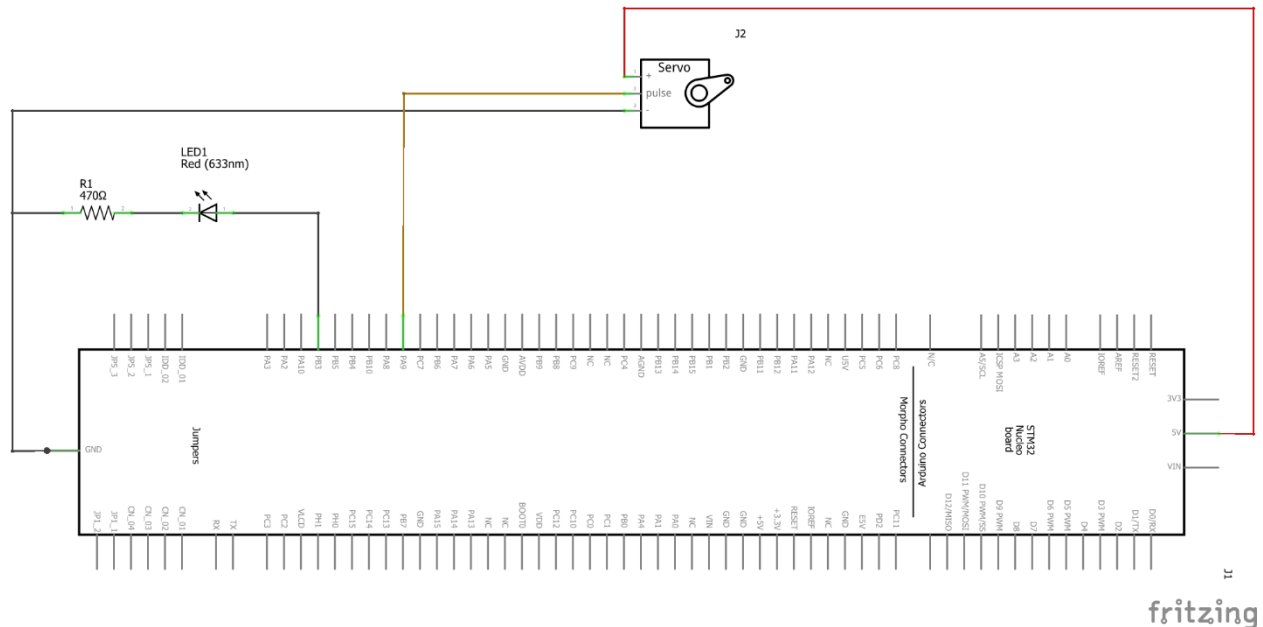


Figure 2. 4. 1 PWM Servo wiring diagram.

And as what is shown in the Figure 2. 4. 1 above, the servo is using **pin 9** of **GPIO port A** of the system. This means that the setup must be prepared for GPIOA register. And by looking at the datasheet, it is possible to use Timer 1 or Timer 2 for this **PA9** GPIO pin. But in this case, **Timer 2**, and **Channel 3**, which connects the GPIO pin to **AF10** is being chosen.

And based on the reference manual, Channel 3 is included in AFRH (Alternate Function Register High). And to be written in the code, this means that the configuration will use register **AFR[1]**. And based on the Figure 2. 4. 2 below, the value of **AF10** in AFRH is **0b1010**.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR15[3:0]				AFR14[3:0]				AFR13[3:0]				AFR12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR11[3:0]				AFR10[3:0]				AFR9[3:0]				AFR8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRy[3:0]**: Alternate function selection for port x pin y (y = 8..15)

These bits are written by software to configure alternate function I/Os

AFRy selection:

0000: AF0	1000: AF8
0001: AF1	1001: AF9
0010: AF2	1010: AF10
0011: AF3	1011: AF11
0100: AF4	1100: AF12
0101: AF5	1101: AF13
0110: AF6	1110: AF14
0111: AF7	1111: AF15

Figure 2. 4. 2 Alternate Function Register High guide table.

This makes the setup of the system goes like this function in this Figure 2. 4. 3 below.

```

2228 void Servo_PWM_Config() {
2229     //enable GPIOA in RCC register.
2230     RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
2231
2232     //resets the Servo's pin MODE. (PA9)
2233     GPIOA->MODER &= ~GPIO_MODER_MODER9;
2234     //resets the Servo's pin OTYPE to 0 as push-pull.
2235     GPIOA->OTYPER &= ~GPIO_OTYPER_OT_9;
2236     //resets the Servo's pin SPEED.
2237     GPIOA->OSPEEDR &= ~GPIO_OSPEEDER_OSPEEDR9;
2238
2239     //sets the Servo's pin MODE to 0b10 as an alternated output.
2240     GPIOA->MODER |= GPIO_MODER_MODER9_1;
2241     //sets the Servo's pin SPEED to medium.
2242     GPIOA->OSPEEDR |= GPIO_OSPEEDER_OSPEEDR9_0;
2243     //resets the Servo's PUPD register.
2244     GPIOA->PUPDR &= ~GPIO_PUPDR_PUPDR9;
2245
2246     //resets the high alternate function for pin PA9.
2247     GPIOA->AFR[1] &= ~GPIO_AFRH_AFRH1;
2248     //activate alternate function 10 to use timer 2 and channel 3 for PA9
2249     GPIOA->AFR[1] |= (0b1010 << GPIO_AFRH_AFRH1_Pos);
2250 }

```

Figure 2. 4. 3 GPIO setup on PA9 with AFRH register.

As for the timer and channel configuration, nothing is changed that much rather than the channel number. But because the assignment asked to set the period in 50 Hz (20ms), the timer Auto Reload Register must be setup just like this Figure 2. 4. 4 below.

```
184
185 //prescale the system using 16MHz frequency.
186 TIM2->PSC = 16 - 1;
187 //set the PWM period in 20 ms.
188 TIM2->ARR = 20000 - 1;
```

Figure 2. 4. 4 Setting up ARR to 20 ms, representing 50 Hz timer tick.

The system also uses the interrupt setup and handler like the part B assignment. However, the rotation of the servo can be implemented like this Figure 2. 4. 5 below.

```
280 void Set_Rotation_Level(int16_t angle) {
281     int16_t pwm_width = 1500 + (angle * 10);
282     TIM2->CCR3 = pwm_width;
283 }
```

Figure 2. 4. 5 Solution to set the PWM output of the servo due to its angular scale.

And the “pwm_width” can be defined in the interrupt handler to be -90°, 0°, or 90°.

3. Experiment Result

By testing the PWM servo on the logic analyzer, such values can be seen in these images below.

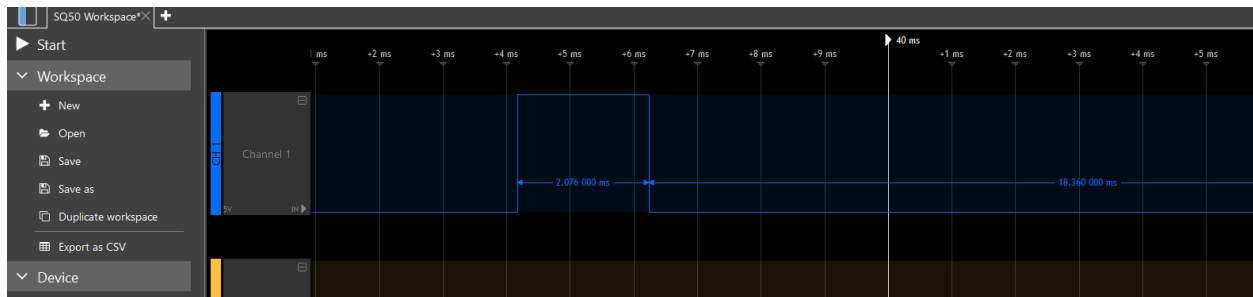


Figure 3. 1 First PWM result on Servo Pulse.

This Figure 3. 1 above is the first rotation of the servo from 90° to -90° angular scale which takes about 2 milliseconds. After that, the system will wait for 20 milliseconds before doing the second rotation.

Then, shown in this Figure 3. 2 below, the system rotates the servo from -90° to 0° angular scale which takes roughly 1.5 milliseconds.

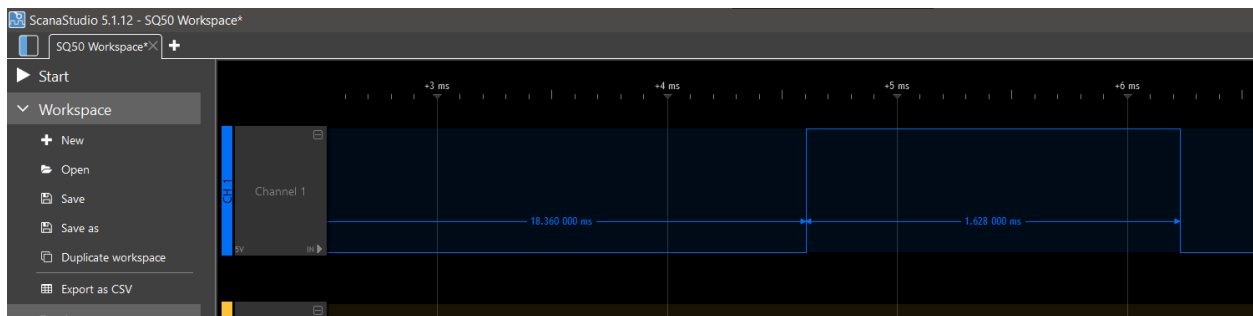


Figure 3. 2 Second PWM result on Servo Pulse.

And after 20 milliseconds later, the system rotates the servo again, from 0° to 90° angular scale within 1 millisecond, 0.5 millisecond faster than the previous one as what is shown in this Figure 3. 3 below.

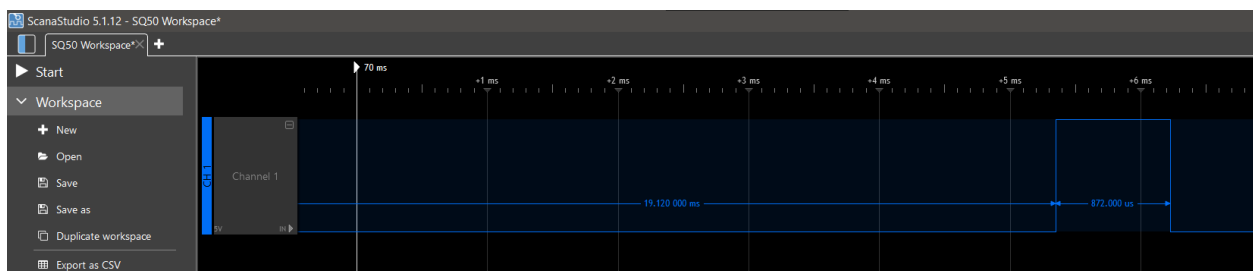


Figure 3. 3 Third PWM result on Servo Pulse.

4. Conclusion

While doing this assignment and some testing, it is known that using the GPIO timers is much more useful than depending the timer with `millis()` counter or `delay`. The system can use the timer register to spot a moment when it must give an exact amount of output within a period without needing any delay function. And from this program, the output result of the system can be figured out numerically, without depending on digital value. The system is basically capable to do analog output while using the Timer and alternate function registers on the Nucleo board.

5. Recommendation

It is recommended to use different channel and Timer per pin in each used GPIO port in case the user need to use multiple tools to act as a timer output because if the system use multiple pins in one Timer and channel for multiple tools for different purposes, the timer would not read the interrupt flag and reload period correctly which can make the system would not be functioned correctly.

6. References

Along this practicum, such documents and references are being used.

- Es_03_lab_03 report, June 2024
- Cortex- M4 Devices Generic User Guide (ARM), 2010
- DM00043574-ReferenceManual_STM32F303RE(www.st.com), January 2017
- UM1724 USER manual STM32 Nucleo-64 boards (MB1136) (www.st.com), April 2019
- Stm32f303re_Datasheet (DocID026415 Rev 5) (www.st.com), October 2016
- Arduino Pull-up Pull-down Resistor (https://arduinogetstarted.com/faq/arduino-pull-up-pull-down-resistor?utm_content=cmp-true)
- Resistor Color Code Calculator (<https://www.digikey.com/en/resources/conversion-calculators/conversion-calculator-resistor-color-code>)

Git link to access the source file of the assignment:

- Timers output:

<https://git.fhict.nl/I424848/t-3-pers-spring-24-farros-ramzy/-/tree/main/es/3-timers>